Solution: Flood Fill

Problem Statement

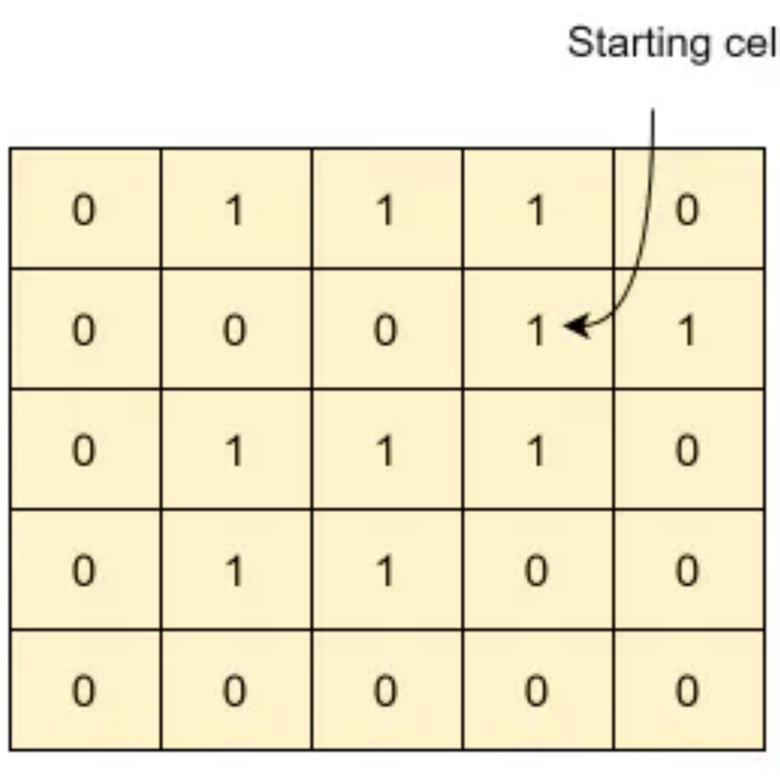
Any image can be represented by a 2D integer array (i.e., a matrix) where each cell represents the pixel value of the image.

Flood fill algorithm takes a starting cell (i.e., a pixel) and a color. The given color is applied to all horizontally and vertically connected cells with the same color as that of the starting cell. Recursively, the algorithm fills cells with the new color until it encounters a cell with a different color than the starting cell.

Given a matrix, a starting cell, and a color, flood fill the matrix.

Example 1:

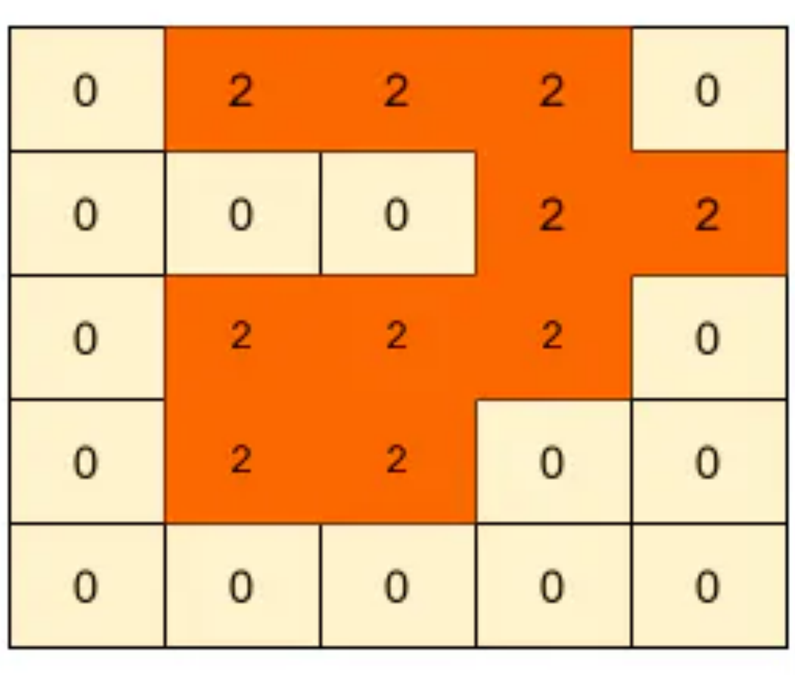
Input: matrix =



0	1	1	1	0
0	0	0	1	1
0	1	1	1	0
0	1	1	0	0
0	0	0	0	0

starting cell = (1, 3)
new color = 2

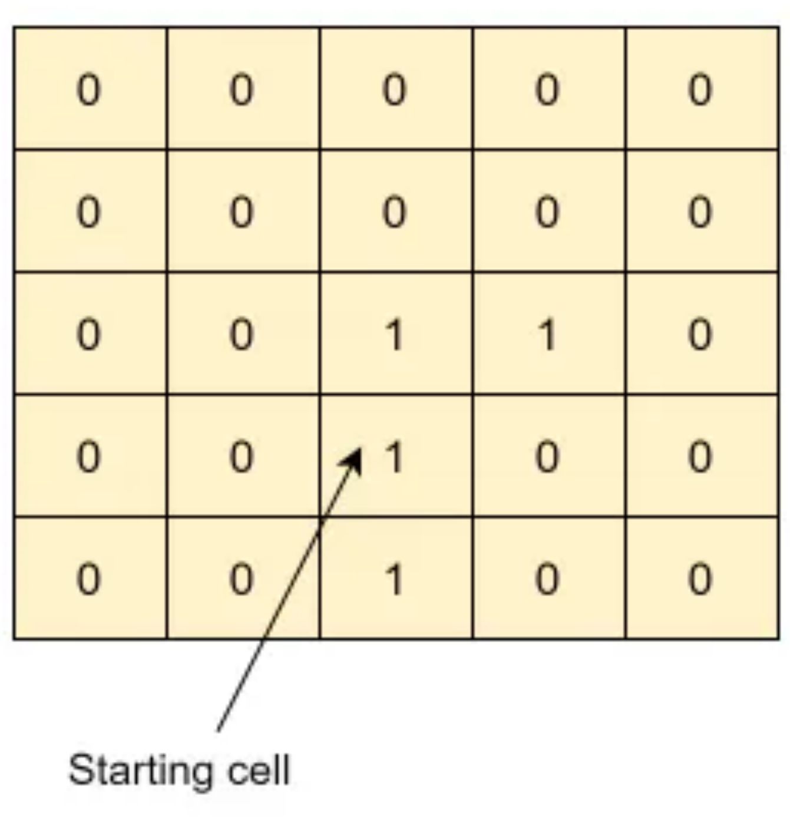
Output:



0	2	2	2	0
0	0	0	2	2
0	2	2	2	0
0	2	2	0	0
0	0	0	0	0

Example 2:

Input: matrix =

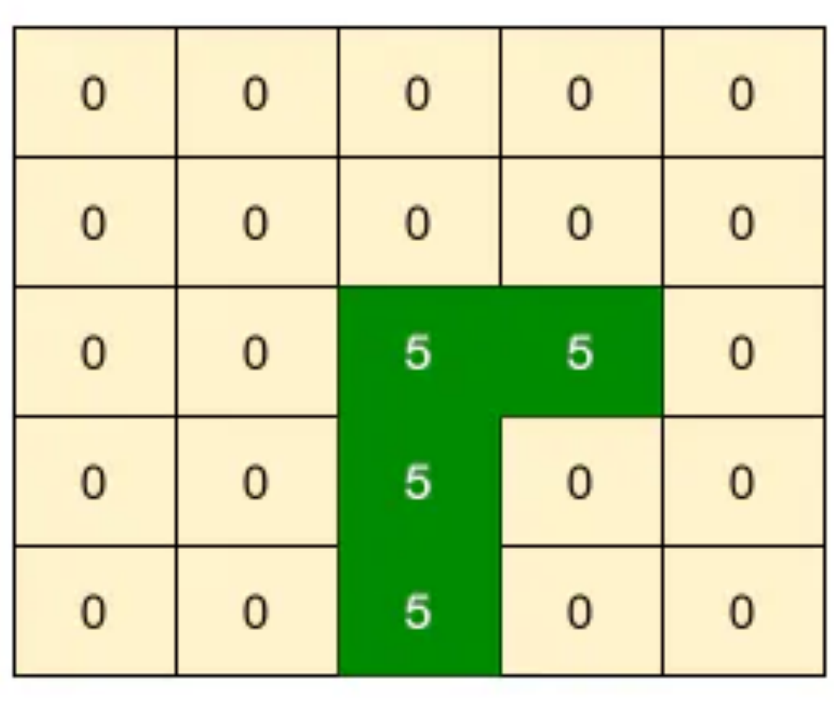


0	0	0	0	0
0	0	0	0	0
0	0	1	1	0
0	0	1	0	0
0	0	1	0	0

starting cell = (3, 2)

new color = 5

Output:



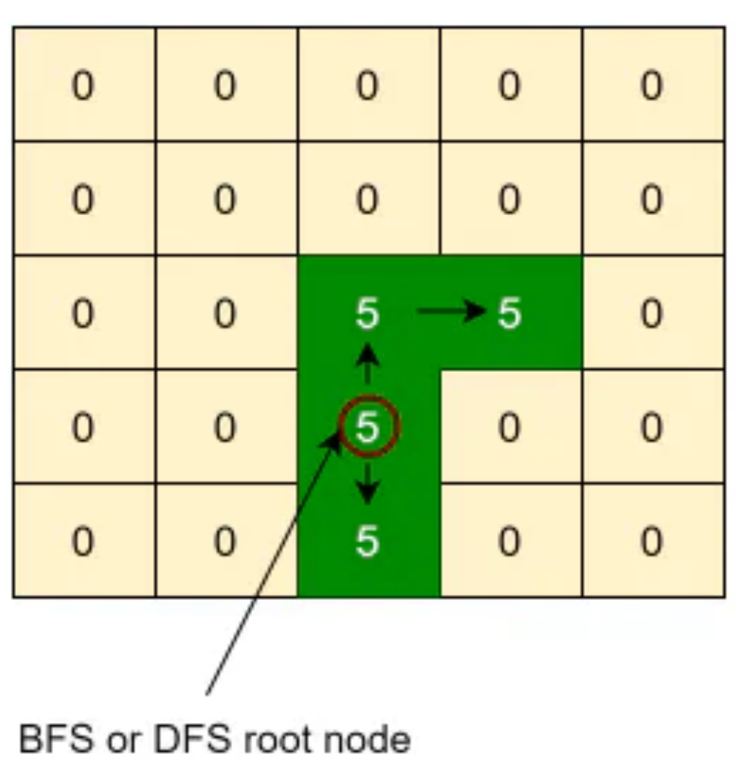
0	0	0	0	0
0	0	0	0	0
0	0	5	5	0
0	0	5	0	0
0	0	5	0	0

Solution

The question follows the Island pattern and is quite similar to Number of Islands problem.

From the given starting cell, we can perform a **Depth First Search (DFS) or Breadth First Search (BFS)** to find all of its connected cells with the same color. During our DFS or BFS traversal, we will update the cells with the new color.

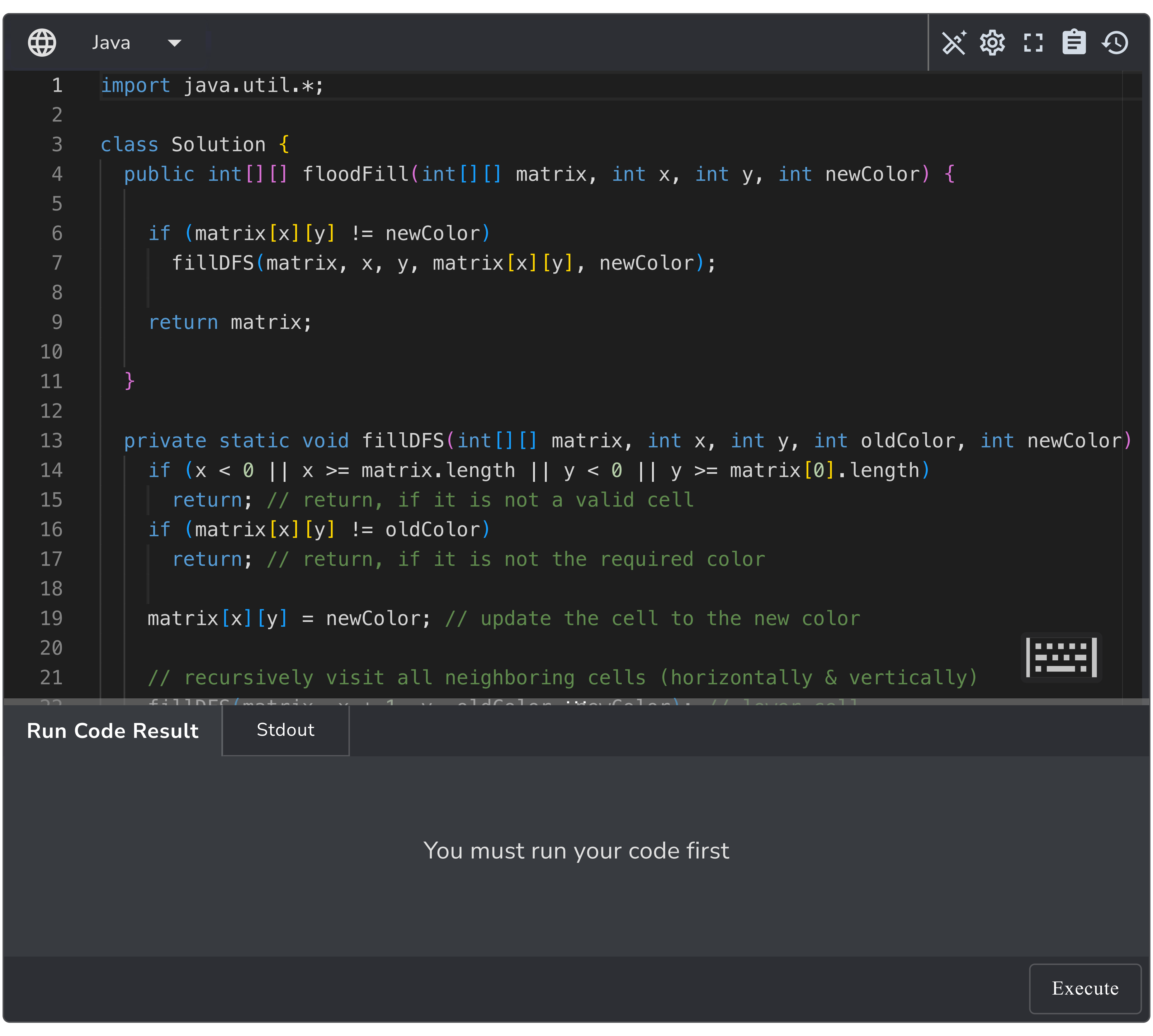
Following is the DFS or BFS traversal of the example-2 mentioned above:



0	0	0	0	0
0	0	0	0	0
0	0	5	5	0
0	0	5	0	0
0	0	5	0	0

Code (DFS)

Here is what our DFS algorithm will look like:



```java
import java.util.*;

class Solution {
  public int[][] floodFill(int[][] matrix, int x, int y, int newColor) {

    if (matrix[x][y] != newColor)
      fillDFS(matrix, x, y, matrix[x][y], newColor);

    return matrix;

  }

  private static void fillDFS(int[][] matrix, int x, int y, int oldColor, int newColor)
    if (x < 0 || x >= matrix.length || y < 0 || y >= matrix[0].length)
      return; // return, if it is not a valid cell
    if (matrix[x][y] != oldColor)
      return; // return, if it is not the required color

    matrix[x][y] = newColor; // update the cell to the new color

    // recursively visit all neighboring cells (horizontally & vertically)
```

You must run your code first

Time Complexity

Time complexity the above algorithm will be $O(M*N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix. This is due to the fact that, in the worst case, we might have to fill the whole matrix.

Space Complexity

DFS recursion stack can go $M*N$ deep when we have to fill the whole matrix. Hence, the space complexity will be $O(M*N)$, where 'M' is the number of rows and 'N' is the number of columns of the input matrix.

← Previous
Flood Fill (Easy)

Next →
Number Of Closed Islands (Easy)

☐ Mark as Completed